

Device Features — Documentation & Logic Overview

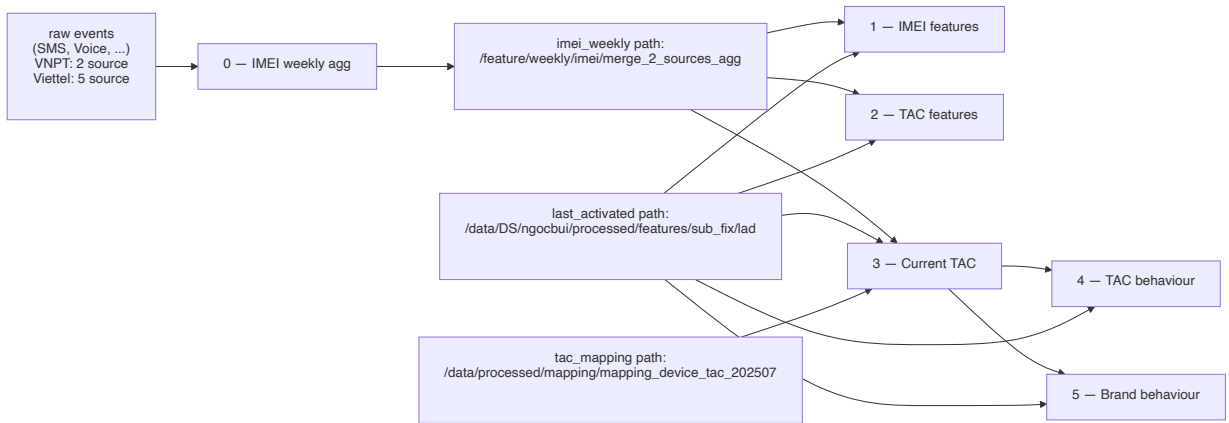
0. Tổng quan

Mục tiêu chung: Sinh ra các feature mô tả hành vi thiết bị (device) cho từng `phone_number` từ dữ liệu weekly IMEI/TAC, phục vụ cho model downstream.

Pipeline gồm **1 bước upstream + 5 bước feature**, mỗi bước ghi parquet về một thư mục riêng theo `snapshot_date` (snapshot luôn là ngày chủ nhật):

#	Script	Output path	Đặc trưng
0	<code>0_build_imei_weekly_aggregation.py</code>	<code>IMEI_WEEKLY_PATH</code>	Tổng hợp raw event (SMS/Voice/...) thành weekly snapshot per phone×imei×tac
1	<code>1_build_device_imei_features.py</code>	<code>DEVICE_IMEI_FEATURES</code>	Feature theo IMEI (số serial máy)
2	<code>2_build_device_tac_features.py</code>	<code>DEVICE_TAC_FEATURES</code>	Feature theo TAC (8 số đầu IMEI → định danh model)
3	<code>3_build_device_current_tac_features.py</code>	<code>DEVICE_CURRENT_TAC_FEATURES</code>	Xác định thiết bị hiện tại (rank 1) & thiết bị thứ 2 (rank 2) trong 2 tuần gần nhất
4	<code>4_build_device_tac_behaviour_features.py</code>	<code>DEVICE_TAC_BEHAVIOUR_FEATURES</code>	Hành vi đổi TAC qua thời gian (switch, streak, gap)
5	<code>5_build_device_brand_behaviour_features.py</code>	<code>DEVICE_BRAND_BEHAVIOUR_FEATURES</code>	Hành vi đổi brand qua thời gian

Quan hệ phụ thuộc giữa các pipeline:



Pipeline 0 chạy hàng tuần để cập nhật snapshot mới. Các pipeline 1-5 dùng kết quả tổng hợp đó như nguồn input chính.

1. Common utilities

1.1 `configs.py` — Các tham số chính

```
SNAPSHOT_DATE_STR = "2024-09-29" # ngày cắt; truyền vào qua sys.argv[1]
WINDOW_WEEKS = [12, 24, 52, 104] # các window tính feature
SWITCH_GAP_WINDOWS_WEEKS = [12, 24, 52, 104]
CURRENT_TAC_WEEKS = 2 # window xác định "thiết bị hiện tại"
CURRENT_TAC_RANKS = [
    {"rank": 1, "prefix": "device_current"},
    {"rank": 2, "prefix": "device_2nd"},
]
```

Hàm `configure(snapshot_date_str)` tính các `START_DATE_* = SNAPSHOT_DATE - 104 weeks` (riêng `CURRENT_TAC = - 12 weeks`, `LAST_ACTIVATED = - 2 weeks`).

1.2 agg_columns.py — SQL builder cho window aggregation

Đây là **core utility** dùng xuyên suốt 5 pipeline. Logic:

- `get_lxw_windows([12, 24, 52, 104])` → sinh danh sách window đơn `[(last 12 weeks, ""), (last 24 weeks, ""), ...]`.
- `get_lxw_overlap_windows([12, 24, 52])` → bổ sung **window lịch sử song song** dạng `(last 24 weeks, last 12 weeks)` để so sánh “tập hiện tại” với “tập lịch sử cùng kích thước” → dùng cho `intersection_ratio`, `except_ratio`.
- `build_groupby_window_query(agg_exprs, windows, snapshot_date, group_by, source)` → sinh **một câu SQL duy nhất** với nhiều `FILTER (WHERE date > snapshot - interval)` để tính tất cả các window trong một lượt scan.
- `build_aggregate_columns(prev_columns, stat_funcs)` → sinh expression cho các phép thống kê:
 - **Stat thường**: `min`, `max`, `avg`, `std`, `skewness`, `kurtosis`.
 - **entropy**: Shannon entropy → $-\sum p \cdot \log_2(p)$ với $p = \text{col} / \text{SUM}(\text{col}) \text{ OVER } (\text{PARTITION BY phone_number})$.
 - **intersection_ratio** (Jaccard): $|\text{current} \cap \text{historical}| / |\text{current} \cup \text{historical}|$.
 - **except_ratio**: $|\text{current} - \text{historical}| / |\text{current} \cup \text{historical}|$.

Note query: với mọi dataframe build qua `spark.sql(query)`, có thể `print(query)` để xem SQL chi tiết. Trong các function bên dưới, biến `query` trả về từ `build_groupby_window_query` chính là string đó.

1.3 device_helpers.py — Helper viết SQL

- `register_temp_view(df, name)` → đăng ký temp view để dùng trong SQL.
- `build_ratio_column(num, den, alias)` → ratio an toàn `null/0`.
- `build_datediff_column`, `build_days_since_column` → datediff helpers.
- `build_tac_columns(prefix, suffix)` → chuẩn hoá brand/model + **suy ra OS** (`apple` hoặc model chứa `iphone` → `ios`, else `android`).

2. Pipeline 0 — IMEI weekly aggregation (upstream)

Mục tiêu: Tổng hợp dữ liệu raw event từ nhiều nguồn (SMS, Voice, ...) về dạng **weekly snapshot** per `(phone_number, imei, tac)`, đếm số ngày active trong tuần. Đây là **nguồn input duy nhất** cho 5 pipeline phía sau.

Nguồn dữ liệu raw (cùng schema `phone_number, imei, tac, date`):

Provider	Source events
VNPT	VOICE, SMS
Viettel	VOICE, SMS, DATA CHARGE, HTTPS, LOCATION

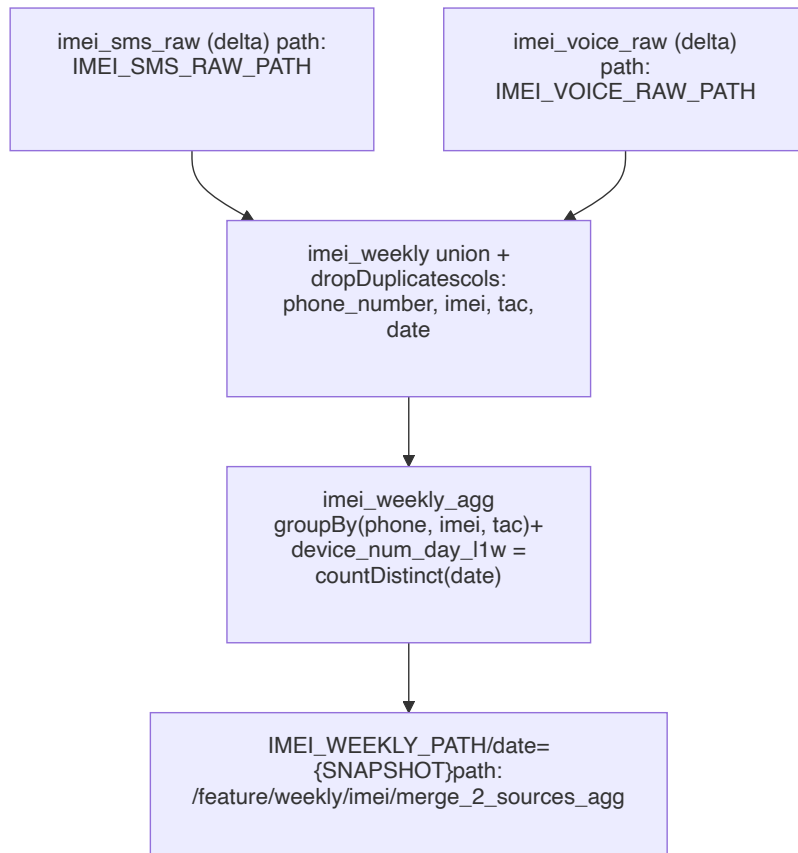
Input / Output:

	Path	Filter
Input	IMEI_SMS_RAW_PATH (delta)	<code>date ∈ [SNAPSHOT - 6d, SNAPSHOT]</code>
Input	IMEI_VOICE_RAW_PATH (delta)	<code>date ∈ [SNAPSHOT - 6d, SNAPSHOT]</code>
Output	IMEI_WEEKLY_PATH/date={SNAPSHOT}	parquet, 1 row/(phone×imei×tac)

- **Cadence:** snapshot luôn là **ngày chủ nhật** → cửa sổ 7 ngày là tuần T2→CN. Pipeline 1–5 đều giả định weekly cadence này.

- Với Viettel, input sẽ có thêm `IMEI_DATACHARGE_RAW_PATH` , `IMEI_HTTPS_RAW_PATH` , `IMEI_LOCATION_RAW_PATH` .

Logic flow:



Logic chính:

- Cửa sổ 7 ngày:** `[snapshot - 6, snapshot]` — gom toàn bộ event trong tuần thành 1 snapshot.
- Union nhiều source:** tất cả source được select về cùng schema (`phone_number`, `imei`, `tac`, `date`) rồi `unionByName + dropDuplicates` . Với Viettel mở rộng, chỉ cần thêm 3 source DATA CHARGE / HTTPS / LOCATION vào chuỗi union (cùng schema).
- device_num_day_l1w** : số ngày distinct trong tuần per (`phone`, `imei`, `tac`) — feature cốt lõi được tái sử dụng ở pipeline 1 và 2 trong các phép `sum(device_num_day_l1w)` để ra **tổng số ngày active** trên window dài hơn (12/24/52/104w).

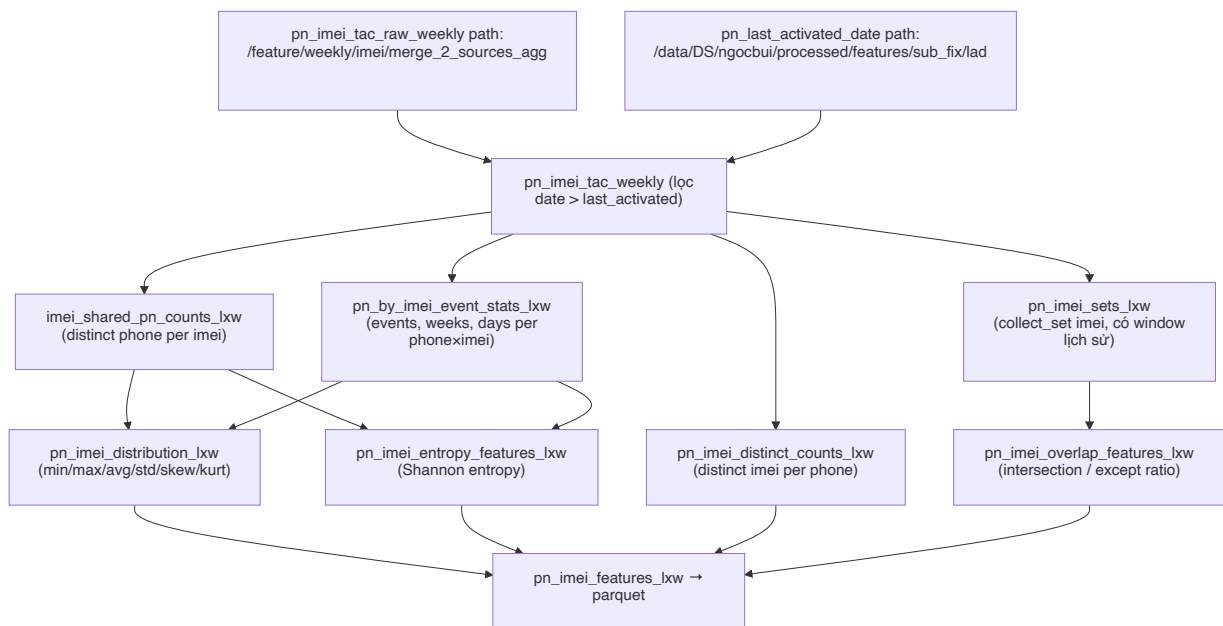
3. Pipeline 1 — IMEI features

Mục tiêu: Với mỗi `phone_number` , tính feature mô tả **tập IMEI** (serial máy) đã sử dụng theo nhiều window — bao nhiêu IMEI, IMEI có được dùng chung với ai, phân bố tần suất, entropy, mức chồng lấp giữa giai đoạn gần và xa.

Input / Output:

	Path	Filter
Input	<code>IMEI_WEEKLY_PATH</code>	<code>date ∈ (SNAPSHOT - 104w, SNAPSHOT]</code>
Input	<code>LAST_ACTIVATED_PATH</code>	<code>date ∈ (SNAPSHOT - 2w, SNAPSHOT]</code>
Output	<code>DEVICE_IMEI_FEATURES/date={SNAPSHOT}</code>	parquet, 1 row/phone_number

Logic flow:



Một số chú ý logic:

- **Lọc `date > last_activated_date`** đảm bảo chỉ tính dữ liệu sau khi thuê bao được kích hoạt (loại nhiều khi số chưa thuộc về user).
- **Distribution stats** loại trừ một số cột không phù hợp qua `stat_config`. Các cột này đã được đánh giá qua thực nghiệm và cho kết quả không tốt trên **toàn bộ stats** hoặc **toàn bộ time window** → loại để giảm chiều feature space. Ví dụ:

```
{"func": "min", "exclude": ['device_pn_imei_events_count', 'device_imei_distinct_pn_count']}
```

- **Overlap features** dùng `WINDOW_WEEKS[:-1]` (bỏ 104w) vì so sánh `1Xw` với `12Xw_to_1Xw` cần $2X \leq 104$.
- **Entropy** được tính 2 bước: tạo column `*_proba_1Xw = col / SUM(col) OVER (PARTITION BY phone_number)`, sau đó group lại để tính `-Σ p · log2(p)`.
- Cuối cùng `F.when(F.isnan(c), None).otherwise(...)` để convert NaN → null cho parquet sạch.

Xem SQL để debug: `build_groupby_window_query` trả về tuple (`query`, `select_sql`) — `query` là full SQL string (gồm SELECT + FROM + GROUP BY), `select_sql` là phần SELECT only (tiện để in các cột mới sinh ra). `build_aggregate_columns` trả về (`columns`, `sub_columns`) cùng tư tưởng. `print(query)` hoặc `print(", ".join(columns))` đều ok.

4. Pipeline 2 — TAC features

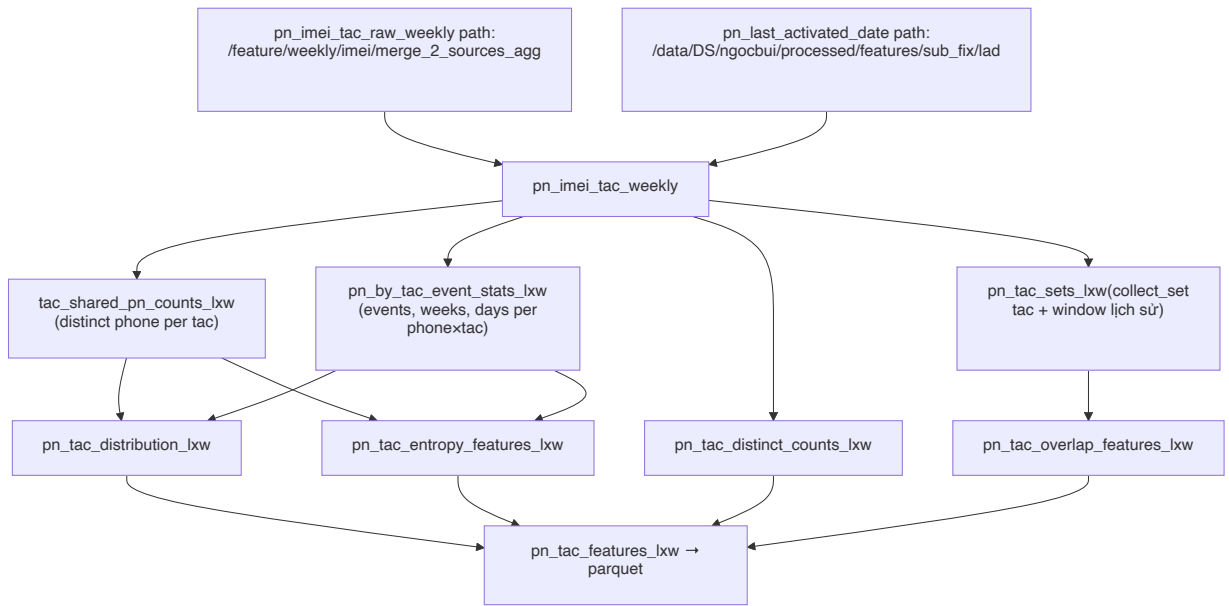
Mục tiêu: Hoàn toàn giống pipeline 1 nhưng group/aggregate theo **TAC** (8 chữ số đầu IMEI — định danh model thiết bị) thay vì IMEI. Một TAC tương ứng với một model → đo lường tần suất, độ đa dạng và overlap ở mức model.

Khác biệt duy nhất so với pipeline 1:

Pipeline 1 (IMEI)	Pipeline 2 (TAC)
<code>group_by=["imei"]</code>	<code>group_by=["tac"]</code>
<code>collect_set(imei)</code>	<code>collect_set(tac)</code>
<code>count(distinct imei)</code>	<code>count(distinct tac)</code>
prefix <code>device_imei_*</code> , <code>device_pn_imei_*</code>	prefix <code>device_tac_*</code> , <code>device_pn_tac_*</code>

Pipeline 1 (IMEI)	Pipeline 2 (TAC)
sum(device_num_day_11w) (lưu ý naming gốc)	sum(device_num_day_11w)
Output: DEVICE_IMEI_FEATURES	Output: DEVICE_TAC_FEATURES

Logic flow:



Vì logic giống hệt pipeline 1, có thể coi đây là **template được tham số hoá theo cột group_by**. Nếu refactor sau này có thể hợp nhất 2 script thành 1 function nhận `key_col ∈ {imei, tac}`.

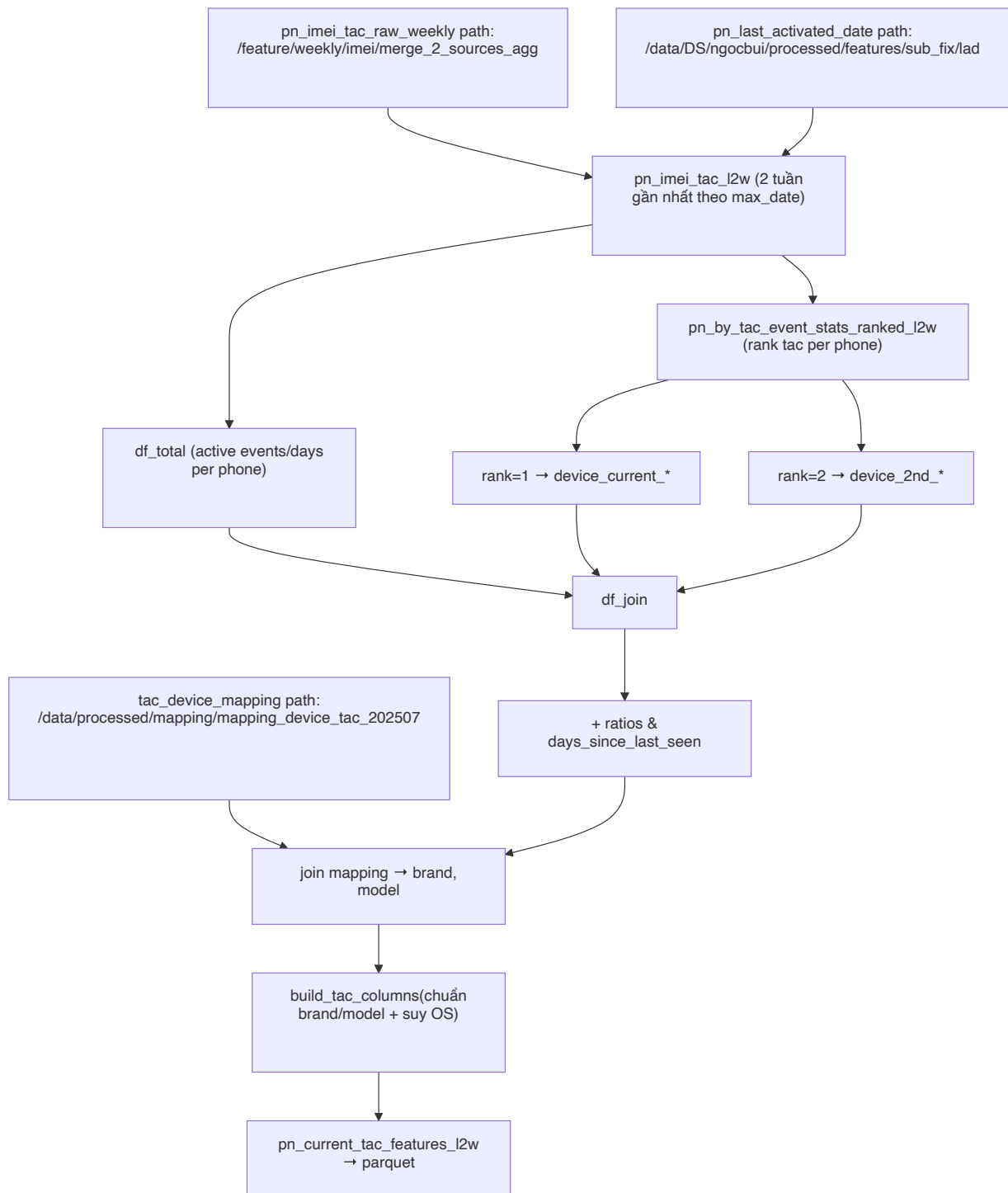
5. Pipeline 3 — Current TAC features

Mục tiêu: Xác định **thiết bị đang dùng** của mỗi `phone_number` tại snapshot (rank 1 = chính, rank 2 = phụ) dựa trên 2 tuần gần nhất, kèm thông tin brand / model / OS và các ratio thể hiện độ "thống trị" của device đó.

Input / Output:

	Path	Filter
Input	IMEI_WEEKLY_PATH	date ∈ (SNAPSHOT - 12w, SNAPSHOT] , imei not null, dropDuplicates
Input	LAST_ACTIVATED_PATH	giống các pipeline khác
Input	TAC_MAPPING_PATH	mapping tac → device_brand, device_model (broadcast)
Output	DEVICE_CURRENT_TAC_FEATURES/date={SNAPSHOT}	1 row/phone_number

Logic flow:



Các logic chính:

- **Định nghĩa "current" theo phone, không theo snapshot:** lấy `max(date)` per phone rồi giữ rows trong 2 tuần gần nhất (`CURRENT_TAC_WEEKS = 2`). Điều này quan trọng với những phone không active đúng tại snapshot.
- **Rank thiết bị:** ưu tiên `active_days desc` → `last_seen_date desc` → `tac asc` (tie-breaker bằng `tac` để deterministic). Rank 1 = `device_current`, rank 2 = `device_2nd` (theo `CURRENT_TAC_RANKS`).
- **Dominance ratio:** `dominant_day_ratio = tac_active_days / device_active_days` và `dominant_events_ratio = tac_events / device_active_events` → đo độ thống trị của device đó trong tổng activity của phone.

- **Suy ra OS** ở bước `build_tac_columns`: `brand = apple` hoặc model chứa `iphone / apple` → `ios`, còn lại → `android`. TAC null → tất cả null; TAC có nhưng không map được → `unmapped`.

Xem SQL: `df_pn_current_tac_features_12w` build query 2 lần qua `spark.sql(query)` — `print(query)` ngay trước mỗi lần gọi để xem chi tiết.

6. Pipeline 4 — TAC behaviour features

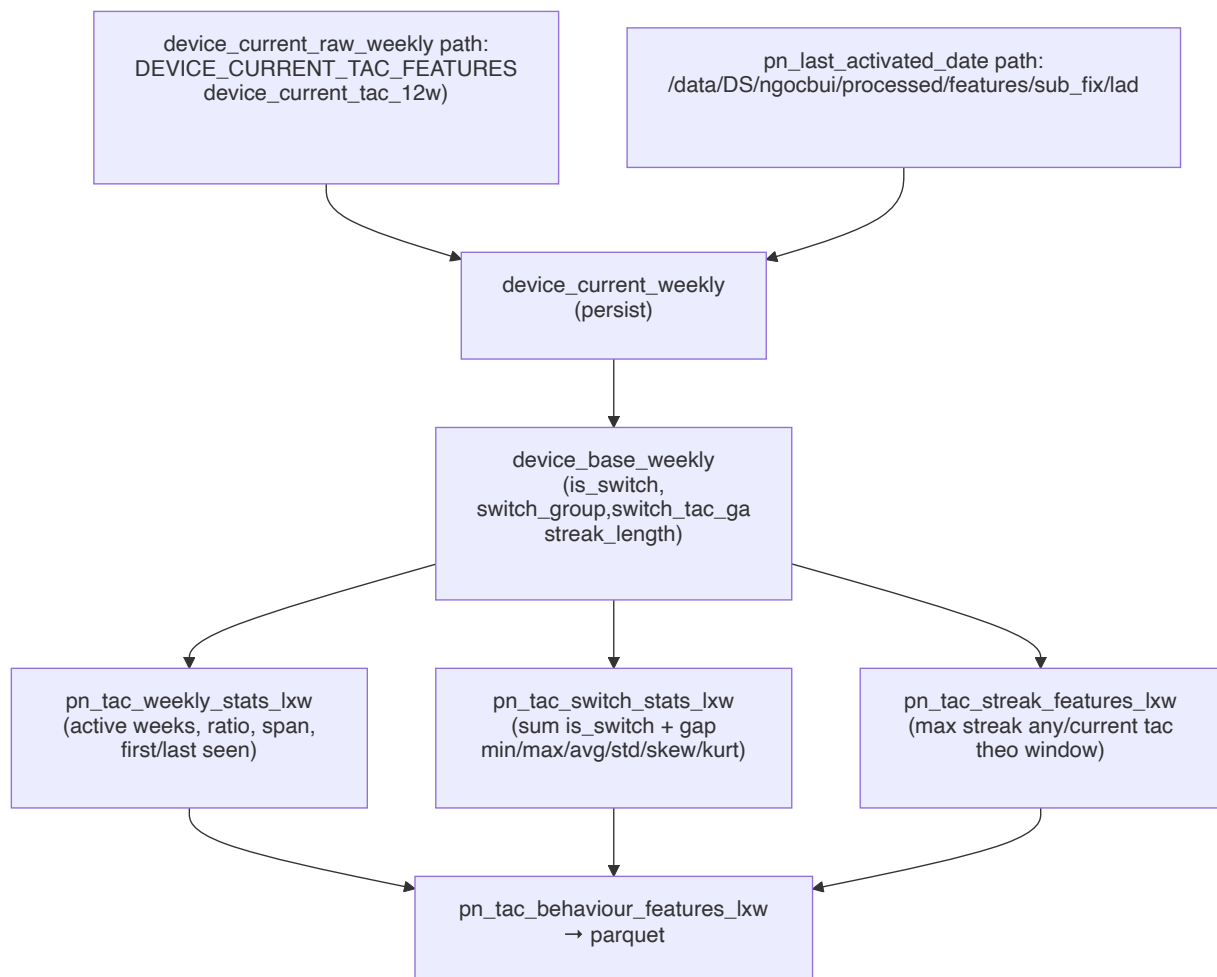
Mục tiêu: Mô tả **hành vi chuyển đổi thiết bị (TAC)** theo thời gian: bao lâu một lần đổi máy, gap trung bình giữa các lần đổi, streak (số tuần liên tục dùng cùng 1 device).

Input / Output:

	Path	Filter
Input	DEVICE_CURRENT_TAC_FEATURES (output của pipeline 3 chạy nhiều snapshot)	<code>date ∈ (SNAPSHOT - 104w, SNAPSHOT]</code> ; lấy <code>device_current_tac_12w</code> → <code>device_current_112w</code>
Input	LAST_ACTIVATED_PATH	dùng làm <code>start_date</code>
Output	DEVICE_TAC_BEHAVIOUR_FEATURES/date={SNAPSHOT}	1 row/phone_number

Lưu ý: cột nguồn là `DEVICE_CURRENT_TAC_COLUMN = 'device_current_tac_12w'` (theo `configs.py`) — script giả định pipeline 3 đã chạy với nhiều `snapshot_date` để tạo lịch sử weekly.

Logic flow:



3 nhóm feature chính (cùng build từ `device_base_weekly`):

1. **Weekly stats** (`pn_tac_weekly_stats_lxw`) — theo từng window 12/24/52/104w: số tuần active, số tuần dùng đúng `current_tac`, số TAC distinct, ngày first/last seen của `current_tac`, kèm các derived: `active_ratio`, `active_span`, `days_since_first/last_seen`.
2. **Switch stats** (`pn_tac_switch_stats_lxw`) — `sum(is_switch)` per window + 6 moments min/max/avg/std/skewness/kurtosis của `switch_tac_gap_lxw` (khoảng cách tuần giữa các lần đổi máy).
3. **Streak features** (`pn_tac_streak_features_lxw`) — group 2 bước qua `switch_group`:
 - `device_max_streak_any_tac_week_lxw`: chuỗi tuần liên tục dài nhất với *bất kỳ* TAC nào.
 - `device_max_streak_current_tac_week_lxw`: chuỗi tuần liên tục dài nhất với *đúng* `current_tac`.

Cuối cùng `pn_tac_behaviour_features_lxw` left-join 3 nhánh trên `phone_number` và chỉ giữ các cột prefix `device_*` (qua hàm `device_cols`).

Các biến state ở `device_base_weekly` (window function):

Cột	Window	Ý nghĩa
<code>current_device</code>	partition phone, order date desc	TAC hiện tại (first non-null từ tuần mới nhất)
<code>prev_device</code>	partition phone, order date	TAC tuần trước → so với hiện tại để detect switch
<code>is_switch</code>	row-level	1 nếu device đổi giữa 2 tuần liên tiếp (cả 2 non-null)
<code>switch_group</code>	partition phone, order date	cumulative sum của <code>is_switch</code> → gom các tuần cùng device
<code>streak_length</code>	partition (phone, switch_group)	độ dài chuỗi tuần liên tục dùng cùng device
<code>switch_tac_gap_lxw</code>	row-level (last sw_col over preceding)	số tuần giữa lần switch hiện tại và lần switch trước trong cùng window (clip về <code>window_start</code> nếu prev nằm ngoài window)

Xem SQL: cả 3 hàm `pn_tac_weekly_stats_lxw`, `pn_tac_switch_stats_lxw`, `pn_tac_streak_features_lxw` đều build query string trước khi `spark.sql(query)` — print biến `query` để xem.

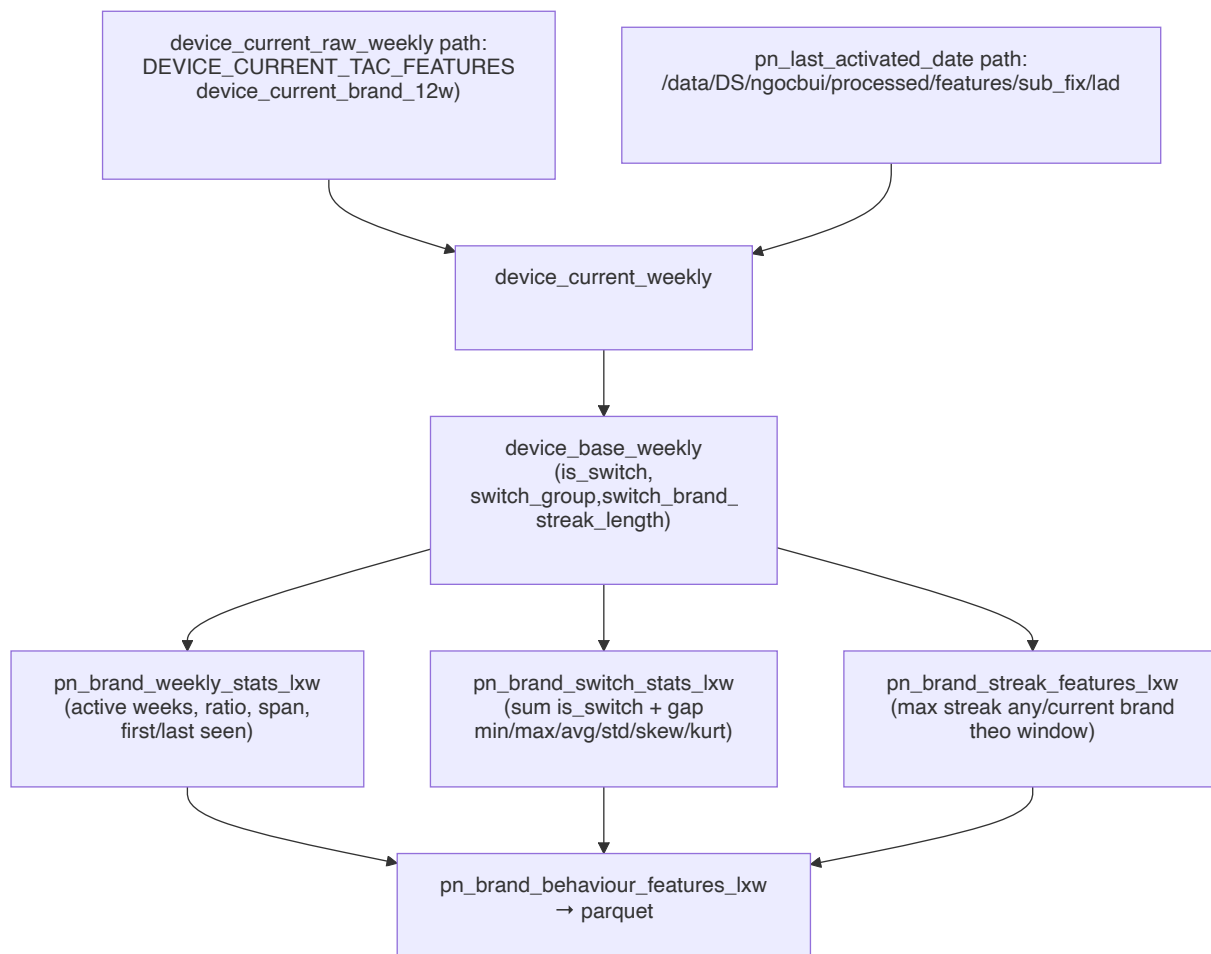
7. Pipeline 5 — Brand behaviour features

Mục tiêu: Hoàn toàn đối xứng với pipeline 4 nhưng đo trên **brand** thay vì TAC. Logic flow, các nhóm feature, biến state đều giống — chỉ khác cột nguồn và naming.

Khác biệt với pipeline 4:

Pipeline 4 (TAC)	Pipeline 5 (Brand)
Cột nguồn: <code>device_current_tac_12w</code>	Cột nguồn: <code>device_current_brand_12w</code>
<code>switch_tac_gap_lxw</code>	<code>switch_brand_gap_lxw</code>
<code>pn_tac_weekly_stats_lxw</code> , <code>pn_tac_switch_stats_lxw</code> , <code>pn_tac_streak_features_lxw</code>	<code>pn_brand_weekly_stats_lxw</code> , <code>pn_brand_switch_stats_lxw</code> , <code>pn_brand_streak_features_lxw</code>
Streak cols: <code>streak_any_tac_week_lxw</code> , <code>streak_current_tac_week_lxw</code>	Streak cols: <code>streak_any_brand_week_lxw</code> , <code>streak_current_brand_week_lxw</code>
Output: <code>DEVICE_TAC_BEHAVIOUR_FEATURES</code>	Output: <code>DEVICE_BRAND_BEHAVIOUR_FEATURES</code>

Logic flow:



Xem SQL: cả 3 hàm `pn_brand_weekly_stats_lxw`, `pn_brand_switch_stats_lxw`, `pn_brand_streak_features_lxw` đều build query string trước khi `spark.sql(query)` — print biến `query` để xem.

8. Cách chạy & checklist

```

# Thứ tự chạy bắt buộc:
python 0_build_imei_weekly_aggregation.py 2024-09-29          # chạy hàng tuần, snapshot là CN
python 1_build_device_imei_features.py 2024-09-29
python 2_build_device_tac_features.py 2024-09-29
python 3_build_device_current_tac_features.py 2024-09-29     # cần chạy nhiều snapshot để có lịch sử
python 4_build_device_tac_behaviour_features.py 2024-09-29
python 5_build_device_brand_behaviour_features.py 2024-09-29
  
```

Checklist khi đổi `snapshot_date` :

- `snapshot_date` phải là **ngày chủ nhật** (pipeline 0 cắt cửa sổ 7 ngày T2→CN).
- Pipeline 0 cần `IMEI_SMS_RAW_PATH` và `IMEI_VOICE_RAW_PATH` có dữ liệu trong khoảng `[snapshot - 6d, snapshot]`.
- Pipeline 1, 2, 3 cần `IMEI_WEEKLY_PATH` (output pipeline 0) có dữ liệu tới `snapshot_date`.
- Pipeline 4, 5 cần `DEVICE_CURRENT_TAC_FEATURES` đã có **nhiều snapshot weekly** trong khoảng `(snapshot - 104w, snapshot]` — nếu mới chạy lần đầu thì lịch sử sẽ rỗng → switch/streak feature sẽ null.

- Tham số `WINDOW_WEEKS`, `CURRENT_TAC_WEEKS` đổi sẽ ảnh hưởng tên cột output → cẩn thận khi join với downstream.

9. Phụ lục

Hai utility chính đều trả về **tuple 2 phần tử**, nên có thể chọn xem đầy đủ hoặc chỉ cột mới:

Hàm	Return	[0]	[1]
<code>build_groupby_window_query(...)</code>	<code>(query, select_sql)</code>	Full SQL (<code>SELECT ... FROM ... GROUP BY</code>)	Chỉ phần SELECT (các cột mới sinh)
<code>build_aggregate_columns(...)</code>	<code>(columns, sub_columns)</code>	List cột chính (vd: <code>entropy, ratio</code>)	List cột phụ (vd: <code>*_proba_lxw</code> của <code>entropy</code>)

```
# Ví dụ với pipeline 1:
from common.src.common.agg_columns import build_groupby_window_query, get_lxw_windows
import configs.configs as config

query, select_sql = build_groupby_window_query(
    agg_exprs=[("count(distinct phone_number)", "device_imei_distinct_pn_count")],
    windows=get_lxw_windows(config.WINDOW_WEEKS),
    snapshot_date=config.SNAPSHOT_DATE_STR,
    group_by=["imei"],
    source="df_pn_imei_tac_weekly",
)
print(query)          # full SQL để chạy thử trên Spark
print(select_sql)     # chỉ xem các cột mới (gọn hơn khi review)
```

Hoặc dùng Spark `explain` để xem plan thực thi:

```
df_pn_imei_features_lxw().explain(mode="formatted")
```